

模組化「引擎 + 可換 spec」元件層

一條凍結的共用介面、九份可換 spec、四個校準-analytic 引擎——把模擬器補成完整、可換型號，每個數字都誠實標註，沒有 silicon 就不假裝有 gate。

2026-06-06 engine + swappable spec no fake gate 所有數字皆可從 committed JSON 重產

Phase 1.2 — 模組化「引擎 + 可換 spec」元件層

Phase 1.1 把自家 **Metis silicon** 量得到的核心 decode 路徑（CIM 算 + 記憶體搬）校到量測級可信。Phase 1.2 把模擬器補成完整、可換型號的元件層：每一個非-micro-benchmark 單元都改寫成「一個模型引擎 + 一份可換的 **spec** 檔」——換型號 = 換 **spec** 檔，引擎程式碼不動。

一條凍結的共用介面

所有單元共用同一個介面（`simulator/models/engine.py`）：

```
Engine(spec, engine='analytic').predict(workload) -> {latency_us, bound, provenance}
```

- **spec** 在建構時綁定，`predict()` 只吃一個 `Workload`（op + 形狀）。
- 回傳 dict 的 key 凍結成 `{latency_us, bound, provenance}`（`bound ∈ {compute, memory, floor}`）。
- 有輕/重兩種引擎的單元（記憶體、NPU）預留 `engine=`：Phase 1.3 把 **Ramulator2** / **ONNXim** 這些 C++ 重型 sim 用 `engine='ramulator2'|'onnxim'` **drop-in** 插進來，不必改 API。

這條介面先用一個 dummy 引擎寫成 conformance test（`tests/test_engine_iface.py`），在平行開發之前就跑綠——它是四個元件各自 fill-in 的合約。

這一期交付什麼

章	單元	引擎	校準狀態
C	CPU (RK3588 big.LITTLE)	指令數 roofline <code>max(compute, memory)+overhead</code>	calibrated (對 fp32 <code>cpu_ops.json</code> , per-op 殘差中位數 1.15%)
N	NPU (RKNPU2)	解析 systolic-roofline	simulated (無 silicon , issue #13 ; 趨勢借 HeteroInfer)
M	記憶體 + SRAM	全-spec analytic (LPDDR4/4x/5) + CACTI SRAM tier	mix : LPDDR4x 24.2 = 量測錨點 ; LPDDR5 = simulated ; 峰值 = assumption
G	GPU (Mali-G610)	解析 roofline 換型號槽 (與 micro-benchmark 並存)	simulated (roofline 下界 , FP16 校準 ; INT8 零資料)
CIM-Card	CIM 重驗	同一顆 AIPU 在量產卡上重量	calibrated (Alpha 13 點) + Card 重驗 (見該章)

誠實標註紀律 (VS PHASE 1.1 一致)

每個數字都帶 provenance 標籤：**calibrated** (對我們手上的 silicon 擬合) / **simulated** / **assumption** / **borrowed** 。最重要的一條鐵律是 **no fake gate**——沒有 **silicon** 就不假裝有數值 **gate** 。RKNPU2 (無板) 與 GPU INT8 (零資料) 因此沒有 per-op 數值驗收門檻，只有 trend-shape / 下界的接受準則，並明寫 issue #13 的 silicon gate 是 *superseded-not-satisfied* (被取代、非達成) 。

可重現原則 (沿用)：每張圖都是 build artifact (`tools/plotting/` 一圖一 script，只吃 committed 數據重產)；每份 report JSON 都能由 `tools/analysis/` 的 fit/build script 重生。整合 cross-check：
`tools/analysis/check_phase1_2.py` (載入九份 spec → 餵各引擎 → 驗凍結 key + 標註一致性 + no-fake-gate)。

與 PHASE 1.1 / 1.3 的界線

- **1.1** 的校準路徑不動：CPU/記憶體引擎雖改成 spec-based，但 1.1 的 capstone recompose (8B decode hold-out **9.5%**) 重跑仍是 9.5% (gate 只用 op-profile bytes + vendor tok/s + BW fit，與這些引擎無關)，1.1 的 fit 腳本重生 byte-identical。
- **1.3** (下一期，疊在本期介面上)：ONNXim (NPU) + Ramulator2 (記憶體 LPDDR5) 當更高保真的可換引擎，交叉比對本期 analytic 的單串流趨勢。招牌價值 (多單元競爭、逐 **token** 整機) 在

Phase 2 °

C — M4-CPU：支援算子的「指令數 **roofline**」（校準到 **fp32 silicon**）

這一章你會學到：為什麼 CPU 上那些「不是大矩陣乘法」的小算子（RMSNorm、RoPE、softmax、SwiGLU、residual、argmax 取樣）也要算進去、為什麼 `exp()` 是它們裡面真正花時間的那一個、我們怎麼用一條 **roofline**（取 $\max(\text{算, 搬}) + \text{固定開銷}$ ）把它們一次描述完、以及這條 roofline 是怎麼對手上真 **silicon**（RK3588 A76 核）校準出來的——誤差中位數只有 **1.1%**。

C.1 架構考量：M4-CPU 是誰？為什麼這些「小算子」要算？

異質 SoC 上，重活（GEMM/attention）丟給 NPU/GPU/CIM，但 LLM 一層裡還有一堆非-GEMM 的支援算子：歸一化（RMSNorm）、位置編碼（RoPE）、注意力的 softmax、FFN 的 SwiGLU 閘、殘差相加（residual）、最後輸出時對 vocab 取 argmax（greedy 取樣）。這些在 profile 裡被指派給 **CPU**（RK3588 的 big.LITTLE：4×A76 + 4×A55）。

它們單看每個都很小（decode 一個 token 約 1–400μs），但每層每 **token** 都要做、層數又多，加總起來在 decode（memory-bound、每個大算子本來就不快）裡並不可忽略。所以 M4-CPU 要能給每個支援算子一個時間，而且要能隨「模型大小、kv 長度、精度、核數」變動——這樣端到端模擬器才接得起來。

C.2 SPEC：CPU_RK3588（每欄位帶 PROVENANCE + HONESTY TAG）

引擎吃一份可換的 **spec**（`simulator/specs/cpu_rk3588.json`），所有硬體數字連同 provenance 都在裡面，換型號 = 換這份檔案：

欄位	值	PROVENANCE / TAG
clusters	A76 4 核 @2.3GHz IPC=2、A55 4 核 @1.8GHz IPC=1	RK3588 TRM [measured]
NEON	128-bit, fp32 4 lanes / fp16 8 lanes	ARMv8.2-A [measured]
cache	L1d 64KiB/核、L2 512KiB/核、L3 3MiB 共享	A76 TRM [measured]
cache_bw_GB/s	L1 73.6、L2 36.8、L3 18.4 GB/s (/核)	A76 TRM 推估 [assumption]
calibration_basis	單 A76 核、單緒、numpy fp32	Phase 0.3 protocol [measured]

⚠️ 兩個邊界寫進 **spec**、本章嚴守：(1) cache BW 是 A76 自己的 SRAM 推估值（**assumption**），不是 Metis AIPU 的 SRAM tier、不是量產卡的 LPDDR4x——WP-CPU 只依這份 spec，不依 WP-MEM。(2) 校準基準是單 **A76** 核單緒；多核是把核數加進去外推，A55（IPC=1）與多核都標 **simulated**。

C.3 原理：一條 ROOFLINE，不是一張查表

前一版（Phase 1.1）這個模組是查表——把 cpu_ops.json 的每個 (model, dtype) 量測值存成常數（issue #10：用量測值不用 FLOP）。能用，但換不了型號：問「IPC×2 的核會多快」「8 核並行多快」「換更大的 vocab」它都答不出來，因為它只記得量過的那幾個點。

Phase 1.2（D1）改成 **instruction-count roofline**——把每個算子拆成「要做多少次運算」和「要搬多少 bytes」，再除以硬體的吞吐：

```
latency_us = max(compute_us, memory_us) + overhead_op
compute_us = (n_elem * ops_per_elem) / (Σ_assigned[W·IPC·freq]) / η_c
memory_us = working_set_bytes / (BW_tier(working_set) · η_bw)
```

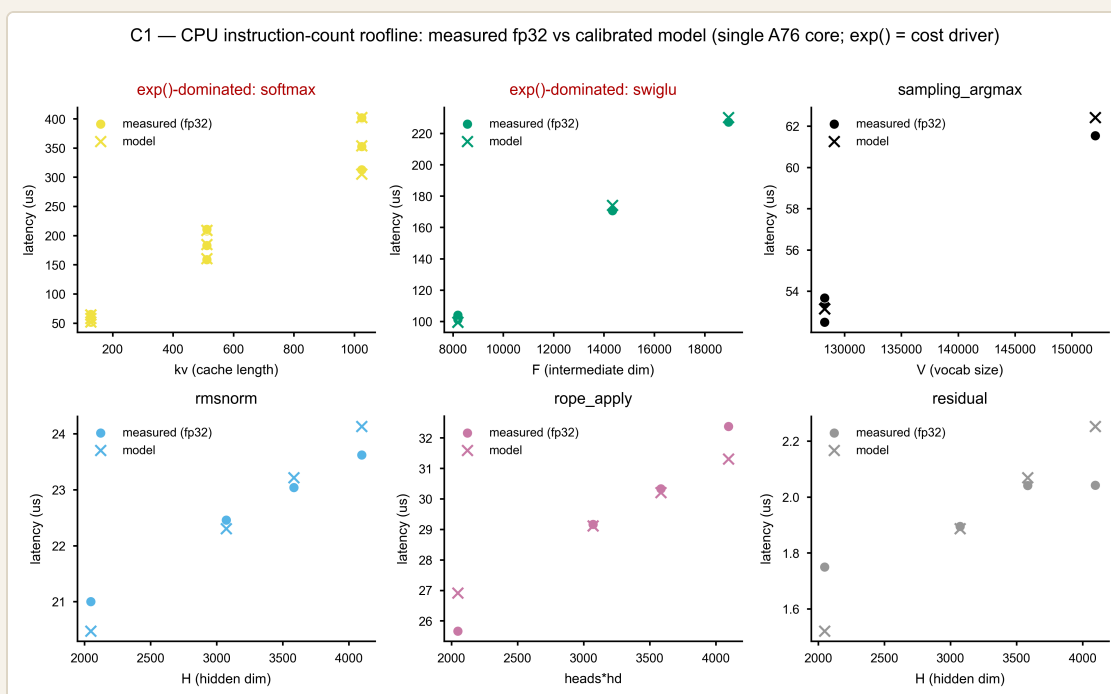
逐項白話：

- **n_elem**（這個算子處理幾個元素）：每個算子的「尺寸變數」——softmax = heads·(kv+1)、swiglu = F（FFN 中間維）、sampling = V（vocab）、rmsnorm/residual = H（hidden）、rope = heads·hd。
- **ops_per_elem**（每個元素要做幾次運算）= 成本主因在這裡：**exp()** 是 **transcendental**（超越函數），一次 **exp** 在 **numpy fp32** 上展開成約 **30** 個融合浮點運算。softmax 和 swiglu 都含 **exp()**，所以它們的 **ops_per_elem** 給很大（≈30+）；residual/argmax 只是一次加法/比較（≈1）；rmsnorm/rope 介於中間（5–6）。我們不靠「**reduction vs elementwise** 二分」——**exp()** 才是真正的軸。（這是結構性假設 **assumption**，不是 fit 出來的。）

- **$\Sigma_{\text{assigned}}[W \cdot \text{IPC} \cdot \text{freq}]$** （指派核的算力）：一個 A76 核的 NEON fp32 吞吐 = 4 lanes × IPC 2 × 2.3GHz = 18.4 G lane-op/s。多核就把核數乘進去——這就是「加核會變快」的來源（外推、simulated）。
- **η_c** （實際達成率，校準）：numpy 的 kernel 不會跑滿 NEON 峰值，實測只到 **≈15.2%**（ $\eta_c=0.152$ ）。這是對 **fp32 cpu_ops.json** 校準出來的唯一全域算力因子。
- **BW_tier**（搬資料走哪一層快取）：依工作集大小選 L1/L2/L3——decode 的工作集（幾 KB 到 ~600KB）全部落在快取裡，永遠不碰 **host LPDDR**（「換 LPDDR4→5 重算」只對 prefill）。
- **η_{bw}** （快取頻寬達成率）= **assumption**，不是校準：見 C.5 的誠實說明。
- **overhead_op**（每個算子的固定開銷，校準）：rmsnorm/rope/residual/argmax-dispatch 在 decode 尺寸下是常數主導的，這個 per-op 固定成本把它吸收掉。

C.4 圖 C1：量測 FP32 VS 校準模型

圖 C1（M4-CPU roofline）— 6 個子圖、每圖一算子、X=尺寸變數、Y=μs



- 怎麼看：實心圓 = 量測（fp32 中位數，4 個模型各一點）；× = 模型在同一形狀下的預測。兩者幾乎重疊。
- 紅標題的 **softmax / swiglu = exp()** 主導：它們的斜率（每多一個元素多花的時間）≈ **12 ns/elem**——兩個算子斜率一致，正是「exp() 是同一個成本來源」的鐵證。softmax 沿 kv（128→1024）線性上升、swiglu 沿 F 線性上升，模型都抓得很準（中位 0.8% / 1.9%）。

- **sampling_argmax**：掃 vocab 的 reduction，4 個點裡 qwen (V=152K) 最大——它的工作集 594KiB 超過 **L2**、落到 **L3**，所以它走的是 **memory (cache)** 分支而非 compute（圖上其餘 3 個 128K vocab 走 compute）。這就是 roofline 的 `max()` 真的在切換的地方。
- **residual**：最小、最吵的算子（見 C.5）。

C.5 SIM-VS-MEASURED（誠實對照）

校準強度逐項（對 **fp32 cpu_ops.json**，單 **A76** 核）：

算子	中位誤差	最大誤差	BOUND	註
softmax	0.8%	2.4%	compute	<code>exp()</code> 主導，斜率 ≈ 12 ns/elem
swiglu	1.9%	4.3%	compute	<code>exp()</code> 主導，與 softmax 同斜率
sampling_argmax	1.1%	1.4%	compute + memory	qwen vocab→L3 走 cache 分支
rmsnorm	1.5%	2.5%	compute	常數主導，overhead 16.8μs
rope_apply	1.9%	4.9%	compute	常數主導，overhead 22.5μs
residual	5.8%	13.1%	compute	見下
整體	1.15%	13.1% (p95 7.3%)		

誠實標註（honesty discipline）：

- **CPU = calibrated**：`η_c` 與 `overhead_op` 對 **fp32 cpu_ops.json** 校準。整體中位誤差 1.15%。
- `exp()` 的 `ops_per_elem`、`byte-passes = assumption`：指令數的物理估計，不是 fit 出來的（fit 的只有 `η_c` 和 per-op overhead）。
- `η_bw = assumption`，不是校準：這份 fp32 decode 資料裡沒有任何「純頻寬解析」的算子——每個算子的工作集都進得了 L1/L2/L3，而且每個都是 compute-bound 或 overhead-bound；加上 repo 裡沒有 **CPU mem-BW micro-benchmark**（稽核缺口）。所以 `η_bw=0.6` 是文獻常見的快取效率佔位值，memory 項只在最大工作集（qwen vocab→L3）才真的綁住，而那一點的量測佐證（不否認）這個假設（誤差 1.4%）。它的存在是為了 prefill / 架構研究的更大工作集。
- **residual** 誤差 **5.8%**（最大 **13.1%**）在它自己的量測噪音內：residual 是所有算子裡最吵的（cpu_ops.json 的 cov 高達 **0.25**，即 25% 量測抖動），值本身只有 1.75–2.04μs（接近 `perf_counter` 解析度、量化成幾個離散階）。它是 **overhead** 主導（固定 floor $\sim 0.8\mu s$ ），模型誤差小於它自己的量測噪音。

- **fp16 = simulated** 上界：fp16 在 A76 上是 **numpy** 模擬（非原生）→ 視為上界。引擎的 roofline 校準在 fp32；fp16 路徑回傳同一條 compute roofline（provenance 字串標明「fp16 = emulated UPPER BOUND」）。**swiglu fp16** = 混精度（silu 的 **exp** 走 fp32）。
- **A55 / 多核 = simulated**：校準基準是單 A76 核單緒；多核把核數加進 Σ （外推）、A55 用 IPC=1。provenance 字串在非單-A76 路徑會附「A55/multicore EXTRAPOLATED = simulated」。
- **NO FAKE GATE**：沒有發明數值 gate。校準項報的是對 **fp32 silicon** 的 **per-op** 殘差（真實數字）；非校準項（ η_{bw} 、fp16、A55/多核）都明標 assumption/simulated。

C.6 可換性 (ENGINE + SPEC)

- 共用介面：`CpuModel(spec, engine='analytic')`、`predict(wl)` 回凍結 dict `{latency_us, bound, provenance}`（`bound ∈ {compute, memory, floor}`）——與其他單元引擎同簽名。換 CPU 型號 = 換 `simulator/specs/*.json`（clusters / cache / BW），引擎碼不動。
- 架構研究就緒：因為是 roofline 不是查表，可直接問「IPC×2」「8 核並行」「更大 vocab/kv」「更大 L2」這類 what-if——把 `cores / cluster` 放進 `Workload.extra`，或改 spec 的 cache 容量/BW 即可。
- 便利路徑：保留 `op_us(op, model, dtype='fp16', kv=None)`（回 `predict(...).latency_us`），讓 recompose 端到端的接線只需 ~2 行。
- 校準腳本：`tools/analysis/fit_m4_cpu_instrcount.py` 解 η_c / `overhead_op`、報 per-op 殘差 → `validation/reports/phase1.2/m4_cpu.json`；產出的因子寫進 `params/m4_cpu_instrcount.json`，引擎讀它。

一句話總結 C：CPU 支援算子用一條 **max(算, 搬) + 固定開銷** 的 roofline 描述，**exp()**

(softmax/swiglu) 是真正的成本主因（同一條 ~12 ns/elem 斜率）； η_c （~15% 峰值達成率）與 per-op overhead 對 **fp32 真 silicon** 校準（整體誤差中位 1.1%）， η_{bw} 、fp16、A55/多核都誠實標為 assumption/simulated；換型號只換 spec、引擎不動。

N — M4-NPU：RKNPU2 解析 systolic-roofline (全模擬，無 silicon)

這一章你會學到：為什麼這一整章的 NPU 數字沒有一個是量出來的、我們怎麼用一個「6 TOPS 天花板 + 借來的 32×32 對齊 + order/shape 罰則」的解析模型把 RKNPU2 補成模擬器裡一個可換的槽、這個模型的形狀為什麼能對上 HeteroInfer 的三條趨勢（Fig3 階梯 / Fig4 order/shape / Fig5 頻寬），以及為什麼「對上形狀」不等於「校準到 silicon」。

N.1 架構考量：NPU 是誰？為什麼這章全是模擬？

異質 SoC 的第三顆計算單元是 **NPU**（RK3588 上的 RKNPU2，一個 systolic-array 加速器，和 Hexagon 同類）。在 CIM-centric 的設計裡，它是 compute-bound matmul 的候選承載者——HeteroInfer（SOSP'25）量到「**NPU » GPU for matmul**」，把 Matmul/FFN-down 丟給 NPU、RMSNorm/SwiGLU 丟給 GPU。但我們手上沒有 **RKNPU2 silicon**。aetina 板在 Phase 0.3 期間離線（issue #13），RKNPU2 的 matmul/attention micro-benchmark 從未採集。所以本章和 Phase 1.1 的 CIM/Mali 章節有一個本質差別：

	資料來源	誠實標籤
CIM (M1)、Mali (M4-GPU)	我們自己的 silicon 量測	calibrated
NPU (M4-NPU，本章)	datasheet + HeteroInfer 借來的趨勢	simulated / borrowed

這章的每一個數字都是 **simulated** 或 **borrowed**，沒有一個是 **calibrated**。它存在的唯一理由，是讓異質模擬器有一個可換的 **NPU** 槽——等 silicon（#13）救回、或 Phase 1.3 接上 ONNXim，這個槽會被取代，而不是被「驗證」。

N.2 原理 + 參數：解析 SYSTOLIC-ROOFLINE

模型（INT8 GEMM，M×K×N）由四個項組成，全部來自 datasheet 或借來的趨勢：

```

padded_MACs = ceil(M/sd)·sd × ceil(K/sd)·sd × ceil(N/sd)·sd      # sd = 借來的 32×32
compute_us   = 2·padded_MACs / (6 TOPS) × order_shape_factor     # 6 TOPS = datasheet
memory_us    = weight_bytes / eff_BW                             # eff_BW = Fig5 帶
latency_us   = max(compute_us, memory_us, dispatch_floor)

```

- 對齊 **padding (borrowed 32×32)** : systolic array 是固定大小的方陣；任何不是 32 倍數的維度都要補滿一整排/一整列，那些補出來的 MAC 是浪費。sd=32 借自 **Hexagon** (HeteroInfer §3.2 Fig3)，標 borrowed。這就是 §N.3(a) 的階梯來源。
- **compute 天花板 (6 TOPS INT8)** : RKNPU2 datasheet 峰值，標 assumption。
- **order/shape factor (≤6×)** : 當輸入 activation 相對權重很寬 (大 N:K)，就破壞了 weight-stall paradigm，吞吐掉到 GPU 等級——HeteroInfer §3.2 Fig4 量到最多 6×。我們用一個 [1, 6] 的乘數隨 $\log_2(N/K)$ 線性上升、在借來的 6× 飽和，標 borrowed。
- **memory 天花板** : 權重串流 bytes / eff_BW，eff_BW 取 Fig5 帶的下緣 (pessimistic)。
- **dispatch floor** : 一個小的固定 per-op 開銷，標 assumption (無 silicon → 名目值)。

native attention (attn_bmm) : activation×activation，沒有靜態權重可 stall → 純 compute-bound ($QK^T + S \cdot V$ ，padded 到 32，無 order/shape 罰則、無權重串流)。

dtypes 限制 : 只支援 **INT4/8/16 + FP16** (datasheet；無 BF16/TF32)。無 **RKNPU2 power telemetry** → 能耗不可判 (這章不報任何 energy 數字)。

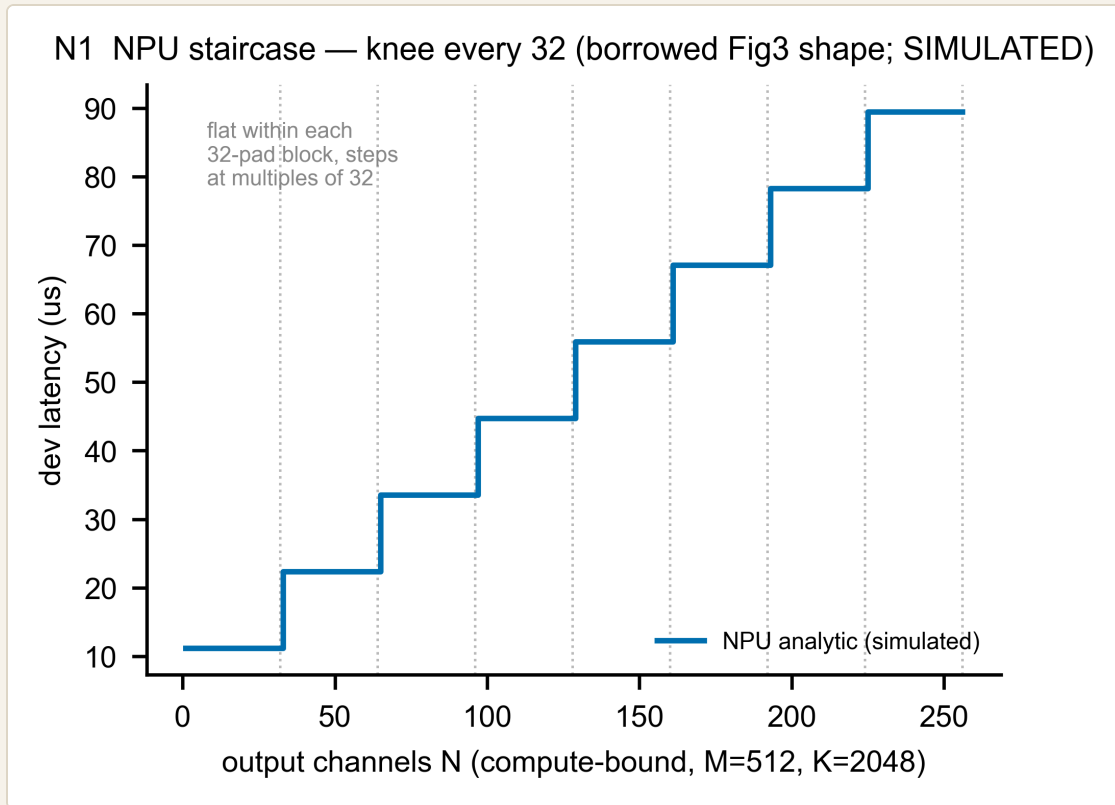
N.3 模擬 VS 參考：三條趨勢勾稽 (這就是驗收)

沒有 **silicon** ⇒ 沒有數值 **gate**。不存在「median ≤10% / p95 ≤20%」這種 per-op 誤差門檻——因為沒有真值可比。唯一的驗收是「形狀」對得上 **HeteroInfer** 的借來趨勢。三條都標 simulated，量化結果寫在 validation/reports/phase1.2/m4_npu.json：

(a) Fig3 階梯：knee 落在借來的 32×32

tools/analysis/build_m4_npu.py 在 compute-bound 區 (M=512, K=2048) 掃 N=1...256，驗：(i) 延遲單調不減、(ii) 每個 32-pad block 內部水平、(iii) 每一階都落在 **32** 的倍數。量化結果：monotone=True、7 個 knee、全部對齊 32。

圖 N1 (NPU 階梯 vs Fig3 形狀) — SIMULATED



- 怎麼看：藍階梯在每個 32-block 內完全水平，到 $N=33$ 、65、97... 就跳一階——這是 systolic array 「不是 32 倍數就浪費一整排」的直接後果，形狀對上 Fig3。
- 誠實邊界：這是借來的形狀，不是 Fig3 的絕對值（我們沒有 Fig3 的數字、也沒有 RKNPU2 silicon）。階梯的高度由 6 TOPS 推得（assumption），不是量出來的。

(b) Fig4 order/shape $\leq 6\times$

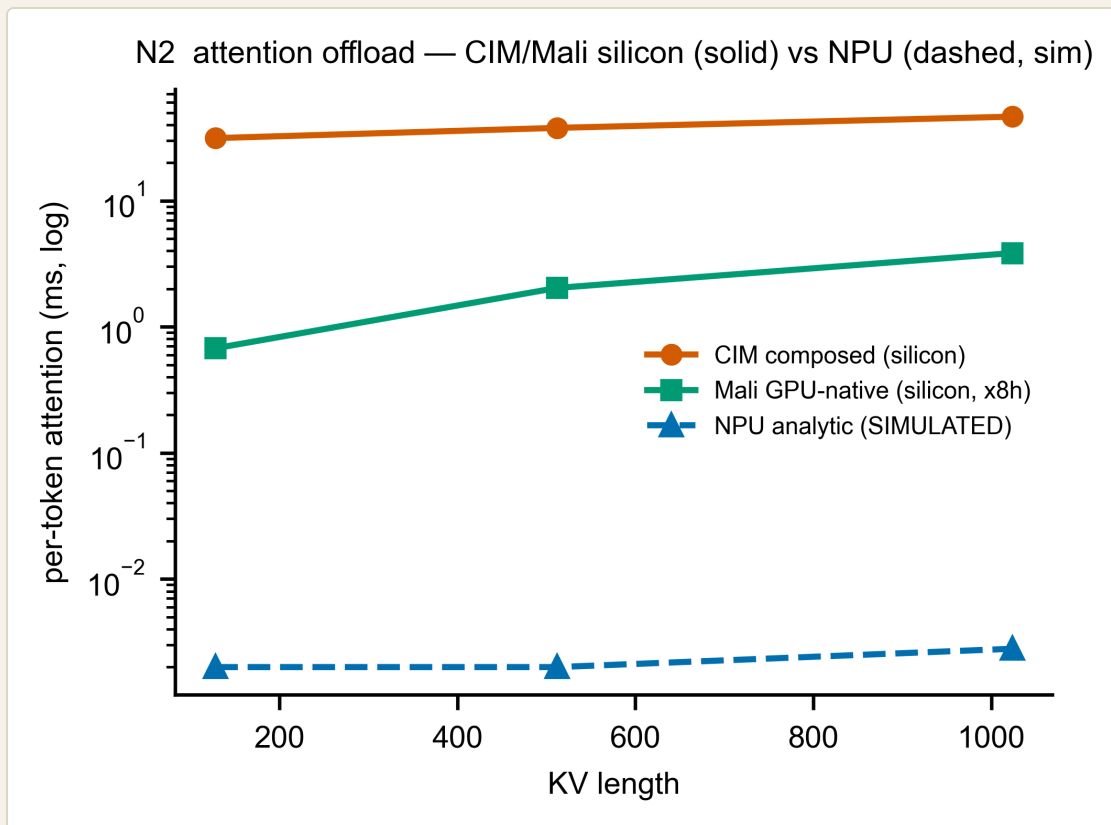
order/shape 乘數在一個寬到 $N=K\cdot 64$ 的 aspect 掃描上，最大值 = $6.00 \leq 6$ （對上 Fig4 的「最多 $6\times$ 」上限）。標 simulated。

(c) Fig5 頻寬 59–66%（分母=68）

有效頻寬帶 = 68 GB/s 峰值的 59–66%（Fig5 量到 single-proc decode 40–45 / 68）。這個 59–66% 的分母是 68，不是 RK3588 的 ~ 34 host BW——這點在 spec 和報告裡都標明（RKNPU2 的絕對帶 $34 \times \text{frac}$ 另存）。量化：frac_low=0.59、frac_high=0.66，落在 $[0.59, 0.66]$ 。標 simulated/borrowed。

attention offload : NPU 是模擬、CIM/Mali 是 silicon

圖 N2 (attention offload) — CIM/Mali 實線=silicon，NPU 虛線=SIMULATED



- **CIM composed** (橘，實線)、**Mali GPU-native** (綠，實線)：Phase 1.1 的 **silicon** deliverable。
- **NPU analytic** (藍，虛線)：本章的解析估計，畫成虛線 + 標 **SIMULATED**，讓誠實邊界一眼可見。
- ⚠ 重要誠實標註：NPU 的絕對值（落在 CIM/Mali 之下約三個數量級）是 6 TOPS 解析天花板的直接輸出——attention 的 FLOPs 對 6 TOPS 來說極小，所以模型算出很快。這和 **HeteroInfer**「**NPU** » **GPU for matmul**」的方向一致，但絕對值未經 **silicon** 驗證，只能當趨勢方向、不可當校準延遲。要把它變成可信的絕對數字，得等 #13 silicon 或 Phase 1.3 ONNXim。

N.4 升級路徑：#13 SILICON VS ONNXIM（兩件不同的事）

升級	是什麼	狀態
issue #13 silicon	真實 RKNPU2 板的 matmul/attention micro-benchmark	superseded-not-satisfied ：未採集（板離線）。本解析模型取代它成為 Phase-1.2 deliverable，但沒有達成那個 silicon gate。#13 的 median/p95 silicon gate 記為 superseded-not-satisfied （ADR-0006 gate 修訂）。
ONNXim (Phase 1.3)	一個通用 systolic NPU 模擬器，接到同一個 engine= 介面後、對這些 HeteroInfer 趨勢交叉驗證	simulated ，不是 silicon 。

關鍵區分：#13 = 真 RKNPU2 **silicon**（缺席）；ONNXim = 更重的模擬器（Phase 1.3），仍然不是 **silicon**。兩者都沒有校準到我們的 **RKNPU2** 板。合約 validation/contracts/m4_npu.yaml 已從 **BLOCKED** 改為 **SIMULATED**，並明寫「無 per-op 數值 gate、驗收=三條 trend-shape 勾稽、#13 silicon gate=superseded-not-satisfied」。

N.5 限制與 GAP（誠實清單）

項目	狀態	說明
全部數字	🟡 simulated/borrowed	6 TOPS + dtypes (datasheet)、32×32 (Hexagon 借)、BW 帶 (Fig5 借)；沒有一個 fit 到 RKNPU2 silicon
數值 gate	❌ 無	無 silicon → 無 per-op median/p95 門檻；驗收=trend-shape 勾稽
#13 silicon	⚠️ superseded-not-satisfied	板離線、micro-benchmark 未採集；解析模型取代非達成 (ADR-0006)
NPU 絕對延遲	⚠️ 未驗證	order/shape 與階梯形狀對上 Fig3/4/5；絕對值未經 silicon，只當趨勢方向
dtypes	✅ 已界定	只 INT4/8/16 + FP16 (無 BF16/TF32)
能耗	❌ 不可判	無 RKNPU2 power telemetry

一句話總結 **N**：RKNPU2 在本期是一個全模擬的解析 systolic-roofline（6 TOPS 天花板 + 借來 32×32 padding + ≤6× order/shape + Fig5 頻寬帶 + native attn bmm），它的形狀對得上 HeteroInfer 的三條趨勢

（這就是 SIMULATED 驗收），但沒有任何一個數字校準到 **silicon**；#13 silicon = superseded-not-satisfied，ONNXim 是 Phase 1.3 的更重模擬器、仍非 silicon。

M — 記憶體（engine + 可換 spec：三種 host DRAM + Metis SRAM tier）

這一章你會學到：Phase 1.2 把記憶體做成「一個解析引擎 + 可換 spec」——同一份 `MemoryModel` 程式碼，換一個 spec 檔（LPDDR4 / 4x / 5、或 CIM 拓樸）就換掉整個記憶體模型；以及為什麼 Metis 晶片上的 SRAM 在 8B 模型下「永遠放不下權重」、所以它是 M1-SPM 的一個「架構假設」層、不是 decode 記憶體牆的解方。

M.1 設計：一個引擎、可換 SPEC (D3 / D5)

Phase 1.1 的 `MemoryModel` 把 PCIe floor、LPDDR、kv_append 寫死在一起。Phase 1.2 改成共用引擎介面（`simulator/models/engine.py` 的 `UnitEngine`）：

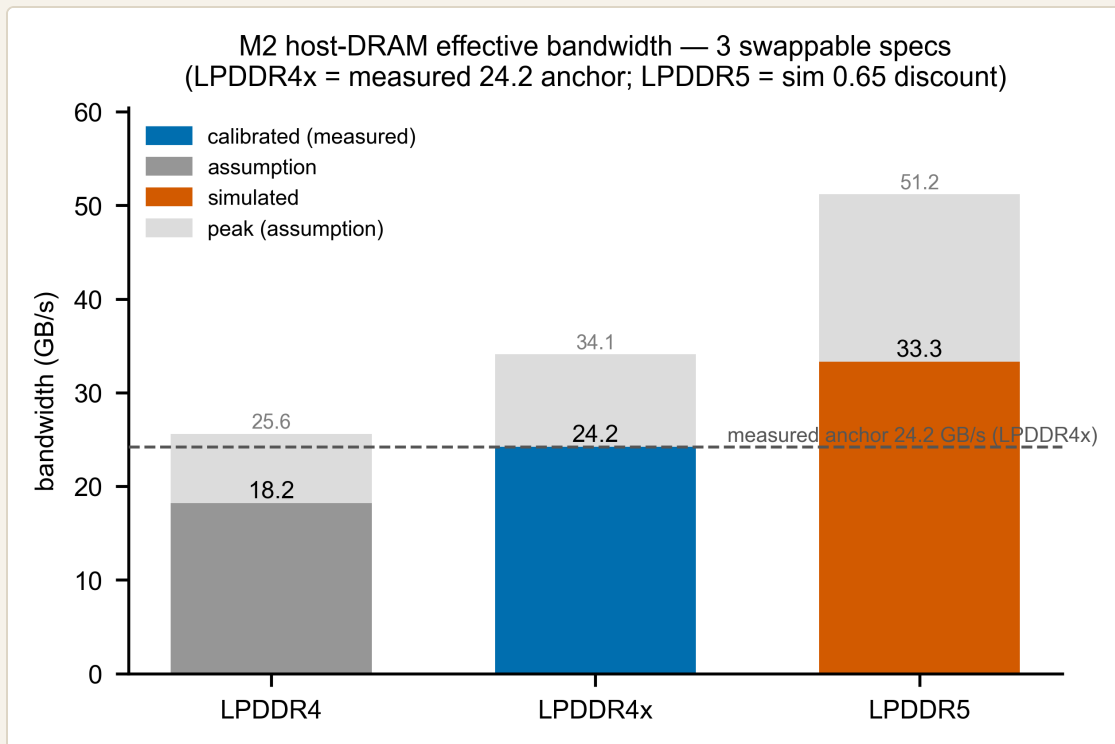
```
MemoryModel(spec, engine='analytic').predict(Workload) -> {latency_us, bound, provenance}
```

spec 建構時綁、`predict` 只吃 `workload`。一份 spec 決定套哪種物理：

SPEC 種類	檔案	模型	BOUND
host DRAM	mem_lpddr4 / mem_lpddr4x / mem_lpddr5	stream/kv = bytes ÷ eff_BW	memory
CIM 拓樸	cim_topo_alpha / cim_topo_card	pcie = per_call_floor + bytes ÷ pcie_BW	floor / memory

換型號 = 換 spec 檔，引擎碼不動。重型後端（`Ramulator2`）走同一個建構簽名，Phase 1.3 插進來（見 M.5）。

圖 M1（三 spec 有效頻寬）



- **X 軸**：三種可換記憶體 spec。淺灰柱：理論峰值（**assumption**——MT/s × 64-bit，in-repo 無 data-rate 出處）。實心柱：有效頻寬，依誠實標籤上色。虛線：量測錨點 24.2 GB/s。
- **LPDDR4x（藍／calibrated）** = 量測錨點：量產 Metis Card 的 on-card LPDDR4x，batch-1 decode 是純權重串流的記憶體牆（decode 時間 ∝ 權重 bytes， $r^2=0.997$ ，voyager-sdk.md:278）→ 有效頻寬 **24.2 GB/s**，= 34.1 峰值的 71%。這是唯一對我們的 **silicon** 校準的數字。
- **LPDDR5（橘／simulated）** = **33.3**：51.2 峰值 × **0.65**。為什麼是 **0.65** 而不是量測到的 **0.71**？LPDDR5 是另一塊記憶體、我們沒有在它上面量過效率——所以保守往下折到 0.65，而不是假設跟 4x 一樣好。這是 simulated，不是 calibrated。
- **LPDDR4（灰／assumption）** = **18.2**：25.6 峰值 × 0.71（把 4x 量到的效率「套用」到 4——一塊不同但相關的記憶體）→ eff_BW 是推導值，標 assumption。

三 spec 都通過 **sanity**：stream latency 對 bytes 正且單調；LPDDR4x→24.2、LPDDR5→33.3（見 `validation/reports/phase1.2/m2_memory.json`）。

M.3 CIM 拓樸：那筆 911MS FLOOR 誰付、誰不付

同一個 `MemoryModel`，餵 CIM 拓樸 spec 就變成 PCIe 傳輸模型：

- `cim_topo_alpha`（量測 **Alpha**）：`per_call_floor_us = 911`（Phase 0.3 量到的 per-call DMA floor，**measured**）+ `bytes ÷ 3.9 GB/s`。每次離散 host↔device 搬移都付這 $911\mu\text{s} \rightarrow \text{bound} = \text{floor}$ 。
- `cim_topo_card`（量產 **Card**）：`per_call_floor_us = 0`——on-card 串流權重不逐次走 PCIe（**architecture**）。

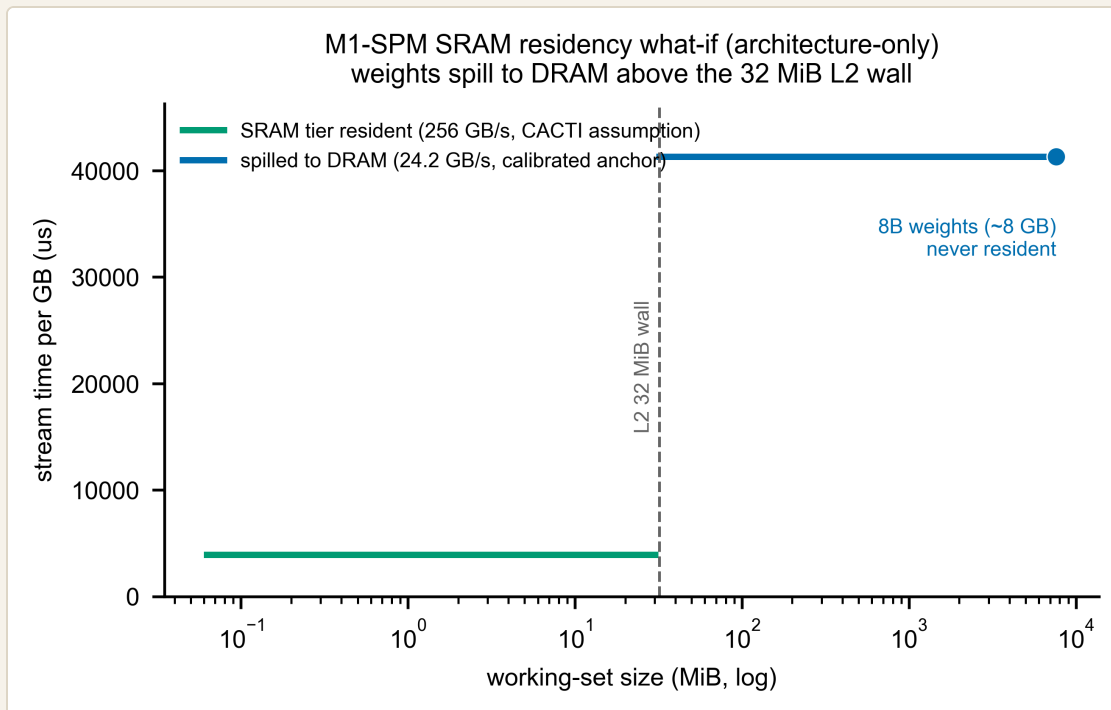
關鍵誠實邊界：911μs 是 Alpha 沒有 **on-card DRAM** 的拓樸特例，不外推到量產卡（比照 Phase 1.1 A2.2）。floor 只對離散搬移（KV-reload、activation handoff、conversion-op）收費，decode 串流權重的主幹用頻寬項。

M.4 METIS SRAM TIER (M1-SPM)：8B 權重「永遠放不下」

SRAM tier 在 **M1-SPM**（`simulator/models/m1_cim_spm.py` 的 `SramTier`，讀 `sram_metis_aipu`），不在 **M2**——M2 管 host DRAM + CIM 拓樸 floor，SRAM 屬 CIM 計算單元的 scratchpad（合約 `m2.yaml` 已註明此歸屬與 ADR-0002 偏離）。

- 容量：L1 4 MiB/核 + L2 32 MiB 共享 + D-IMC 1 MiB/核 = **52 MiB**（datasheet/ISSCC 2024）。
- **BW/latency = CACTI assumption**：Metis 沒有公開 SRAM 的 BW/latency，所以用 CACTI tier 的代表值（256 GB/s、5 ns）當假設。
- **residency = architecture-only**：8B INT8 權重（~8 GB）**> 32 MiB L2**，所以 `predict()` 把這種工作集解析到 **DRAM 層**（永不命中 SRAM）。decode 的記憶體牆是 **host LPDDR**、不是 **SRAM**。

圖 M3 (SRAM tier residency what-if)



- 工作集 ≤ 32 MiB L2 牆 $\rightarrow$ 走 SRAM 層（綠，CACTI assumption 的高頻寬、每 GB 時間低）；超過 $\rightarrow$ **spill** 到 DRAM 層（藍，calibrated 24.2 錨點，每 GB 時間約 10 $\times$ ）。
- **8B** 權重（ ~ 8 GB）的點永遠落在 DRAM 那條線上——這正是「architecture-only」：我們把 SRAM 階層還原進模擬器（之後做架構研究時，「如果權重能放進更大的 SRAM，decode 會快多少」是個 load-bearing 變數），但不宣稱現在的權重放得下。

M.5 為什麼 RAMULATOR2 在 PHASE 1.3、而且 LPDDR4 不在它的預設裡

本期記憶體全是 **calibrated-analytic**；重型 DRAM 模擬（Ramulator2，cacheline 粒度）移到 **Phase 1.3**，走同一個建構簽名 + 凍結 **contract**（`engine='ramulator2'`，ADR-0002 swappable）。這是相對 ADR-0002 原文「Phase 2」的一個時程偏離（已記入 `m2.yaml`）。

⚠ 一個 1.3 要注意的点：LPDDR4(x) 不是 Ramulator2 的一線 DRAM 預設。Ramulator2 內建的標準預設以 DDR3/4/5、LPDDR5、HBM 為主；LPDDR4/LPDDR4x 通常沒有現成 **preset**。所以 Phase 1.3 接 Ramulator2 時，量產卡那塊 LPDDR4x（24.2 錨點）必須自行提供/改寫 **timing config**（或退而用 LPDDR5 preset 做 cross-check，但那是另一塊記憶體、不能當 4x 的驗證）。本期的 calibrated 24.2 錨點正是 1.3 要對齊的目標值。

M.6 限制與 GAP（誠實清單）

項目	狀態	標籤
LPDDR4x 24.2	✅ 量測錨點	calibrated （量產卡 decode 牆， $r^2=0.997$ ）
LPDDR5 33.3	解析	simulated （0.65 折扣 < 量測 0.71，不同記憶體）
LPDDR4 18.2 / 各峰值	推導 / 數學	assumption （套用 4x 效率 / MT/s×64-bit）
Alpha 911µs floor	✅ 量測	measured （Alpha 拓樸特例，不外推 Card）
SRAM BW/latency	假設	CACTI assumption （無公開 Metis SRAM 規格）
residency	架構	architecture-only （8B 權重永不命中 → spill DRAM）
kv_append	未驗證	unvalidated （Phase 0.3 未隔離；純-BW 形式對、待救板校準係數）
Ramulator2	延 1.3	swappable（LPDDR4x 無現成 preset，須自備 timing config）

一句話總結 **M**：記憶體做成「一引擎 + 可換 spec」—— `MemoryModel(spec)` 換 spec 就換型號（LPDDR4/4x/5、CIM 拓樸 A/B），唯一校準的是量產卡 LPDDR4x 的 **24.2 GB/s** 錨點，LPDDR5(33.3)=simulated、峰值=assumption；SRAM tier 在 M1-SPM、CACTI 假設、residency=architecture-only（8B 權重永遠 spill 到 DRAM）；重型 Ramulator2 在 Phase 1.3 插進同一介面（且 LPDDR4x 需自備 timing config）。

G — GPU : Mali-G610 的「解析 roofline 換型號槽」（與 micro-benchmark 並存）

這一章你會學到：為什麼 GPU 在這個 CIM 架構裡是「注意力 offload」的角色、Phase 1.1 量到的那個 micro-benchmark 模型是主力、為什麼這一期又多做一個「解析 roofline」當可換型號的槽、它怎麼用 `max(算, 搬)` 一條式子同時抓住 decode (memory-bound) 和 prefill (compute-bound) 兩個區段、以及一連串不能不講清楚的誠實邊界——尤其是 INT8 在 GPU 上根本零資料、量到的 **20.12** 是 FP16 不是 INT8、峰值 **512** 是假設。

G.1 架構考量：GPU 是誰？為什麼這一期要多一個 ROOFLINE？

回顧整體設計：CIM 負責「投影矩陣 (projection) 那種 $\text{weight} \times \text{activation}$ 」，而 GPU 的角色是「注意力 **offload**」——把 CIM 不擅長的 $\text{activation} \times \text{activation}$ bmm ($\text{QK}^T \cdot \text{S} \cdot \text{V}$) 丟給它。Phase 1.1 已經量了一個 **micro-benchmark** 模型 (`m4_gpu.py` / `MaliGpuModel`)：拿真的 Mali matmul kernel 跑出來的 attn-bmm 斜率 + GEMM 飽和吞吐當下界，這個仍是 GPU 的主力 (**PRIMARY**) 資料源。

那這一期 (Phase 1.2) 為什麼還要再做一個東西？因為這個模擬器的設計是「引擎 + 可換 **spec**」(D5)：每個單元都應該能「換一顆型號 = 換一個 spec 檔、引擎不動」。micro-benchmark 模型綁死在「我們手上這顆 Mali-G610 的實測點」，換一顆 GPU 就沒資料了。所以這一期補一個 **GpuRooflineModel** (解析 **roofline**) 當換型號的槽 (**swap slot**)：它只吃一個 spec (峰值 + 校準效率)，就能對任意形狀給一個 latency。

兩者的關係要講白：**micro-benchmark** = 主力 (實測)，**roofline** = 換型號槽 (解析下界)。不是取代，是並存。`m4_gpu.py` 這一期完全沒動。

G.2 原理 + 參數：一條 `MAX(算, 搬)` 同時抓兩個區段

roofline 的式子就是經典的「算 vs 搬，取較慢者」：

```
latency_us = max( compute_us , memory_us )
compute_us = 2·M·K·N / (eff_compute · fp16_peak)
memory_us = nbytes / mem_eff_BW
nbytes(預設) = (K·N + M·K + M·N) · bytes_per_elem
```

← FLOPs ÷ 有效算力天花板
 ← bytes ÷ 有效頻寬
 ← 權重 + 輸入 act + 輸出 act

兩個校準參數都對 `mali_matmul.json` 的 **FP16** 點校準 (`tools/analysis/fit_gpu_roofline.py`) :

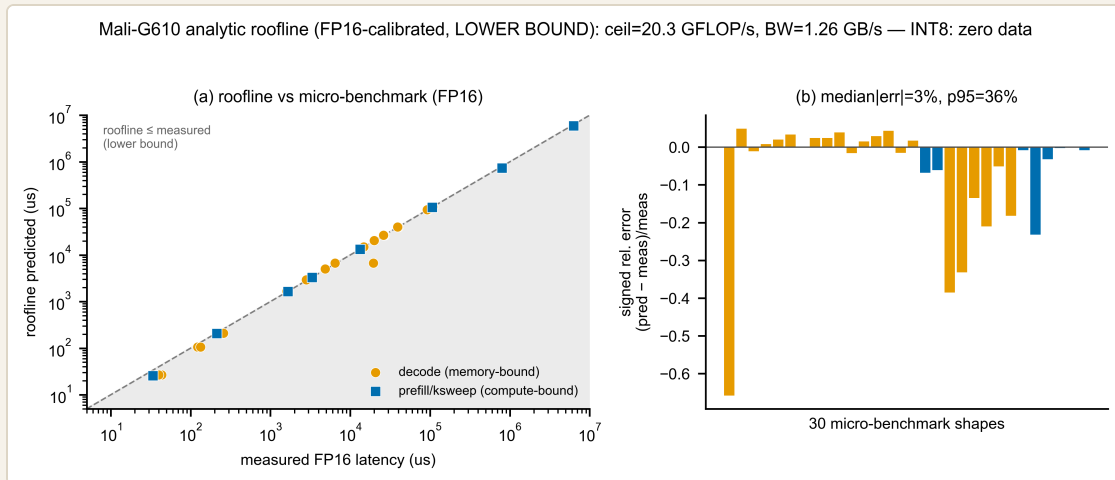
參數	值	怎麼來的	標記
<code>eff_compute_fp16</code>	0.0198	ksweep 飽和 f16 吞吐 20.29 GFLOP/s ÷ FP16 峰值 1024	calibrated (FP16)
<code>mem_eff_BW_GBs</code>	1.256 GB/s	16 個 decode-GEMV 點， <code>lat = nbytes/BW</code> 過原點 <code>numpy.linalg.lstsq</code>	calibrated (FP16)

- 有效算力天花板 = `eff_compute · 1024 ≈ 20.3 GFLOP/s` : 這是 kswep 大方陣飽和後的 f16 吞吐——一個自己寫的、未優化 **OpenCL kernel** 的飽和值，所以它是下界（真正調過的 kernel 會更快）。
- 有效頻寬 $\approx$ **1.26 GB/s** : decode 是 batch-1 GEMV，純搬權重、memory-bound；把 16 個 decode 點的「latency vs 搬的 bytes」用最小平方擬一條過原點的線，反推出有效頻寬。同樣是這顆未優化 kernel 的下界。

為什麼一條 `max` 就夠？因為 decode (M=1 的 GEMV) 會落在 `memory_us` 這一支（搬權重主導）、prefill / kswep 大方陣會落在 `compute_us` 這一支（算主導）—— `max` 自動在兩個區段間切換，`bound` 欄位 (`'memory' / 'compute'`) 會告訴你是哪一支贏。

G.3 SIMULATED VS REFERENCE : ROOFLINE 對 1.1 量測點的誤差

圖 G1 — Mali-G610 解析 roofline (FP16) 對 micro-benchmark 點



- (a) 左圖：X = 實測 FP16 latency，Y = roofline 預測，對角虛線是 $y=x$ 。橘點 = **decode (memory-bound)**、藍方 = **prefill/ksweep (compute-bound)**。灰帶 = $y=x$ 以下，也就是「預測 $\leq$ 實測」的下界區。幾乎所有點都落在對角線上或略低——這正是一個下界該有的方向（預測比實測樂觀一點點、不超過）。
- (b) 右圖：每個形狀的帶號相對誤差 $(pred-meas)/meas$ 。整體 **median |誤差| = 3%**、**p95 = 36%**。大多數點貼著 0；負的長尾來自兩個離群點：一個 $M=1, K=2048, N=2048$ 的 1b decode 點（實測異常慢、-66%），和最小的 $M=64$ kswep（kernel launch 開銷佔比大、-23%）。這些負偏差 = **roofline** 在那兩點比實測更樂觀，與「下界」一致。

怎麼讀這張圖（重點）：median 3% 不是在宣稱「校準很準」——它只是說「這條 **roofline** 抓對了 **decode/prefill** 兩個區段的形狀趨勢，而且坐在實測之下（下界）」。誤差數字記在 `validation/reports/phase1.2/m4_gpu_roofline.json` 的 `error_vs_1p1_measured`（30 點逐點 + median/p95/max）。

G.4 誠實標註（非協商）：這一章最重要的一段

這個模型有一連串不能含糊的邊界，全部標 `simulated`：

項目	狀態	說明
INT8 GPU GEMM	❌ 零資料	Mali matmul kernel 只有 FP32/FP16；INT8 完全沒量。對 int8 workload，predict() 仍用 FP16 校準的天花板，並在 provenance 明標「dtype=int8 has ZERO GPU data; FP16 ceilings used」。
量到的 20.12	⚠️ 是 FP16	Phase 1.1 那個 20.12 GFLOP/s 飽和點是 FP16，不是 INT8。整個 fit、整個 predict 都是 FP16 only。
FP32 峰值 512	⚠️ assumption	spec 的 fp32_peak_gflops=512 是假設，可能低估 2-4x，需驗證。本模型校準對 FP16，所以不依賴這個 FP32 峰值——但 spec 裡仍誠實標 assumption。
roofline 本身	⚠️ 趨勢 + 下界	未優化 kernel + 只有 5 個飽和點 → 不是可轉移的校準 (NOT transferable)。predicted < measured 是預期方向。沒有數值驗收 gate (no INT8 silicon → no fake gate)；驗收 = 趨勢形狀 + 坐在實測之下。
ksweep_saturation_M	🪦 dead param	spec 裡這個欄位是死參數（保留、不刪，依稽核清單）；本引擎用不到。

為什麼沒有 INT8 sim？一句話：因為沒有任何 Mali INT8 GEMM 的量測資料。我們不會憑空編一個 INT8 數字來假裝有 gate——那違反「no fake gate」。要 INT8，得先有 silicon 量測（未來工作）。在那之前，這顆 GPU 在模擬器裡只能給 FP16 的下界趨勢。

G.5 限制與 GAP（誠實清單）

- 可換性已就緒：GpuRooflineModel(spec, engine='analytic') 符合凍結合約 {latency_us, bound, provenance}；換一顆 GPU = 換 roofline_fit 那塊（重跑 fit 或填新峰值）。
- micro-benchmark 仍為主：端到端 recompose / offload 用 m4_gpu.py（實測 attn 斜率），roofline 是換型號槽——兩者並存，職責不同。
- 缺口：(1) INT8 零資料；(2) FP32 峰值 512 待驗證；(3) 5 點飽和 → 不可轉移；(4) 沒有 GPU 記憶體 BW 的獨立 micro-benchmark，mem_eff_BW 是從 decode latency 反推的有效值（含 kernel 開銷）。

一句話總結 G：GPU 這一期多了一個解析 roofline 換型號槽（max(算,搬)，兩軸都對 FP16 校準、是下界），它抓對 decode/prefill 兩個區段的形狀並坐在實測之下（median |誤差| 3%）；但 micro-

benchmark 仍是主力，而且最重要的是——**INT8** 零資料、**20.12** 是 **FP16**、峰值 **512** 是假設、**roofline** 不可轉移，全部誠實標 **simulated**，不編任何 **gate**。

CIM-Card — CIM 計算 kernel 重驗（同一顆 AIPU，未凍結）

為什麼 CIM KERNEL 不該被當成「凍結」

Phase 1.1 的 CIM 計算引擎（M1）校準在 **Metis Alpha** 的 13 個 **native single-tile** 量測點（`simulator/models/params/m1_cim.json`，2D `G_eff(N,K)` 擬合，2.7%）。Alpha 是 pre-production 板、跑不了 **LLM**（封閉韌體 -1301 牆 + 無 on-card DRAM），所以當時無法在 compute-bound regime 多取點。

但同一顆 **quad-core AIPU** 在量產 **Metis Card** 上是活的（`machines.md`：`metis-0:7:0` 16 GiB `clock=800MHz`），而且 `axrunmodel` 的 `dev_fps` 是隔離計算的指標（`dev/system split`，不是合成差分）。兩板都 **800 MHz** → Card 的 `dev_lat` / `dev_gflops` 可以直接對 Alpha 13 點比，不需 **clock** 正規化。

所以 CIM kernel 不是凍結的：我們可以在 Card 上用同一個 `1x1-conv matmul proxy` 重新量測/交叉驗證，並補上 Alpha 量不到的 **prefill** / **compute-bound** 形狀。這把「Alpha 凍結」的疑慮解除，也補了 decode 量不到的計算上界。

方法（已就緒、READY-TO-RUN）

移植 Alpha 的 `characterization/aetina/run_metis_cim.py` 到 Card →
`characterization/metis_card/run_metis_cim_v16.py`：

- 編譯首選低階 `compile`（與 Alpha `run_metis_cim.py:65` 同路徑，1:1）；`compile` 在 v1.6 被移除才退 `deploy.py --mode QUANTCOMPILE`。 `voyager-sdk.md:339` 記「general MatMul 非 `deploy.py` 支援 op (YOLO11 whitelist)」→ 所以 `deploy.py` 只是 best-effort 退路。
- `--spike` 模式（~30 分）先答可行性：低階 `compile` 在不在 + `1x1-conv proxy` 編不編得出 `model.json`。
- 交叉驗證腳本 `tools/analysis/validate_cim_card.py`：Card 的 13 個 alpha-shape `dev_gflops` 對 Alpha 13 點算 `median/p95 |rel_diff|`（同顆 AIPU、800 MHz、無 rescale），另列 `prefill/compute-bound` 新點。

本期狀態：DEFERRED_FALLBACK（誠實標註）

本 session 無法上機：harness 自動把「SSH 進共用 Metis 板」判為未授權動作擋下（量測節點是共用實驗室硬體，需使用者明確授權；當時使用者離線）。我們不繞過這個 **denial**。

這正是 plan 設計好的 fallback 情境（plan step 18 註 + handoff §5：低階 compile 不在 / MatMul 不支援 / 板不可達 → 退「**Alpha 13** 點 **calibrated**（非凍結、待板） + Card e2e 驗 memory-wall」並回報 **user**、不靜默改路徑）。所以：

- **CIM** 計算維持 **calibrated**（**Alpha 13** 點）——這是 Phase 1.1 已有的、量測級可信的校準，不受影響。
- **Card** 重驗延後，狀態寫進 `validation/reports/phase1.2/cim_card_revalidate.json`（`status: DEFERRED_FALLBACK`），含解鎖步驟。
- 移植腳本 + validator 已寫好並語法/執行驗證（validator 跑得出 fallback 報告），只差授權。

解鎖：給 SSH 權限（settings 加一條 `ssh metiscard` 的 Bash 規則）後：`rsync` 腳本上 Card → 跑 `run_metis_cim_v16.py --spike`（先看可行性）→ `full` → `rsync` 拉回 → 重跑 `validate_cim_card.py`。報告會自動從 `DEFERRED_FALLBACK` 變成 `CARD_REVALIDATED`（含 13 點一致性 + prefill 新點）。

一句話

CIM 計算的可信度沒有降級：仍是 Phase 1.1 的 Alpha-13-點量測校準。Card 重驗是升級路徑（解凍 + 補 compute-bound），腳本就緒、待一個授權；在那之前，誠實標成 `DEFERRED_FALLBACK`、回報使用者。